Nanjing University of Information Science and Technology

Laboratory Report

Experiment Title: Distributed Object Storage & Fault Tolerance Testing

Date	2026-06-12	Class	2
Name	whatokk	Student ID	
GitHub Account	whatokk	Repository	whatokk/lab9-distributed-object-storage
Environment	GitHub Actions Ubuntu runner	Storage	MinIO / Docker Compose

一、Experimental Purpose

- Master object storage by understanding bucket/object organization instead of traditional file paths.
- Deploy a distributed MinIO object storage service using Docker Compose.
- Observe metadata and object fragments stored across multiple simulated disks.
- Experience fault tolerance by deleting one storage directory and verifying that data remains readable.

二、Experiment Contents

- Created four independent storage directories: data1, data2, data3, data4.
- Created docker-compose.yml for MinIO server and MinIO Client containers.
- Started cloud-minio and cloud-mc containers.
- Configured mc alias myminio.
- Created bucket my-cloud-storage.
- Generated and uploaded a 20 MiB random object bigfile.bin.
- Observed xl.meta and part fragments across all storage directories.
- Stopped the cluster, deleted and recreated data3, restarted the cluster.
- Verified that bigfile.bin was still accessible and hash-identical after recovery.

- Cleaned up containers and saved experiment evidence.

三、Detailed Steps and Results

Step 1-2 Deploy Directory Structure

Created a dedicated experiment project and four storage directories to simulate four independent storage disks.

Step 3-4 docker-compose.yml Explanation

The minio service uses quay.io/minio/minio, mounts data1-data4 as volumes, exposes ports 9000 and 9001, sets root credentials, and starts distributed storage with server /data{1...4}. The mc service uses minio/mc and depends on minio so it can manage buckets and objects.

Step 5 Launch Cluster

Ran docker compose up -d. docker ps showed cloud-minio and cloud-mc running, with MinIO ports 9000 and 9001 mapped.

Step 6 Configure MinIO Client Alias

Ran mc alias set myminio http://minio:9000 admin password123. The output showed Added myminio successfully.

Step 7 Create Bucket

Created bucket my-cloud-storage with mc mb.

Step 8 Create and Upload Large File

Generated bigfile.bin using dd with size 20 MiB, copied it to the mc container, and uploaded it to myminio/my-cloud-storage/.

Step 9 Verify Upload

mc ls showed bigfile.bin as a 20MiB STANDARD object.

Step 10 Observe Backend Fragmentation

The backend directories contained xl.meta metadata and part.1/part.2 fragments under data1, data2, data3, and data4. This shows MinIO stored object data and metadata across the simulated disks.

Step 11 Simulate Disk Failure

Stopped the cluster, deleted and recreated data3 with sudo rm -rf data3, then restarted the cluster.

Step 12 Verify Fault Tolerance

After the simulated failure, mc ls still listed bigfile.bin. The object was downloaded as recovered-bigfile.bin and its SHA256 hash matched the original exactly.

Step 13 Cleanup

Ran docker compose down and verified with docker ps that no containers were left running.

四、Evidence Screenshots

Lab9 Experiment Run Summary

Captured on Windows 10 | 2026-06-12 01:25:22 +08:00 | GitHub account: whatokk

```
Repository: https://github.com/whatokk/lab9-distributed-object-storage
Workflow: Lab9 Distributed Object Storage
Run URL: https://github.com/whatokk/lab9-distributed-object-storage/actions/runs/27364710248
Status: completed
Conclusion: success
Event: push
Head SHA: bfcc7c68af3dd282775c61dc2546298bf1998e2e
```

Figure 1. Lab9 successful GitHub Actions run.

4-Disk MinIO Cluster Deployment Evidence

Captured on Windows 10 | 2026-06-12 01:25:22 +08:00 | GitHub account: whatokk

```
docker-compose.yml key content:
services:
  minio:
    image: quay.io/minio/minio
    container_name: cloud-minio
    volumes:
      - ./data1:/data1
      - ./data2:/data2
      - ./data3:/data3
      - ./data4:/data4
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: admin
      MINIO_ROOT_PASSWORD: password123
    command: server /data{1...4} --console-address ":9001"
  mc:
    image: minio/mc
    container_name: cloud-mc
    depends_on:
      - minio
    entrypoint: /bin/sh
    command: -c "tail -f /dev/null"
    tty: true

docker ps after launch:
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS
NAMES
98a23f3a5762   minio/mc             "/bin/sh -c 'tail -f…"   21 seconds ago   Up 20 seconds
cloud-mc
35f04037bf62   quay.io/minio/minio   "/usr/bin/docker-ent…"   21 seconds ago   Up 20 seconds
0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp   cloud-minio
```

Figure 2. Docker Compose configuration and running MinIO containers.

Bucket Creation and Object Upload Evidence

Captured on Windows 10 | 2026-06-12 01:25:22 +08:00 | GitHub account: whatokk

```
MinIO client alias:
Added `myminio` successfully.
Bucket creation:
Bucket created successfully `myminio/my-cloud-storage`.
20 MB file:
-rw-r--r-- 1 runner runner 20M Jun 11 17:19 bigfile.bin
Upload result:
`/bigfile.bin` -> `myminio/my-cloud-storage/bigfile.bin`
┌────────────┬──────────────┬──────────┬──────────────┐
│ Total      │ Transferred  │ Duration │ Speed        │
│ 20.00 MiB  │ 20.00 MiB    │ 00m00s   │ 253.05 MiB/s │
└────────────┴──────────────┴──────────┴──────────────┘

Object listing:
[2026-06-11 17:19:36 UTC]  20MiB STANDARD bigfile.bin

Object stat:
Name      : bigfile.bin
Date      : 2026-06-11 17:19:36 UTC
ETag      : 7f1f03c4a3b0c9d1e8568afb5eeeedc-2
Size      : 20 MiB
Type      : file
Metadata  :
```

Figure 3. Bucket creation and 20 MiB object upload.

Backend Storage Fragmentation Evidence

Captured on Windows 10 | 2026-06-12 01:25:22 +08:00 | GitHub account: whatokk

```
Key fragments found across data1-data4:
data1/.minio.sys/buckets/my-cloud-storage/.metadata.bin/xl.meta
data1/.minio.sys/config/config.json/xl.meta
data1/.minio.sys/config/iam/format.json/xl.meta
data1/.minio.sys/pool.bin/xl.meta
data1/my-cloud-storage/bigfile.bin/5df985c3-32f2-4544-a171-179952cd9f23/part.1
data1/my-cloud-storage/bigfile.bin/5df985c3-32f2-4544-a171-179952cd9f23/part.2
data1/my-cloud-storage/bigfile.bin/xl.meta
data2/.minio.sys/buckets/my-cloud-storage/.metadata.bin/xl.meta
data2/.minio.sys/config/config.json/xl.meta
data2/.minio.sys/config/iam/format.json/xl.meta
data2/.minio.sys/pool.bin/xl.meta
data2/my-cloud-storage/bigfile.bin/5df985c3-32f2-4544-a171-179952cd9f23/part.1
data2/my-cloud-storage/bigfile.bin/5df985c3-32f2-4544-a171-179952cd9f23/part.2
data2/my-cloud-storage/bigfile.bin/xl.meta
data3/.minio.sys/buckets/my-cloud-storage/.metadata.bin/xl.meta
data3/.minio.sys/config/config.json/xl.meta
data3/.minio.sys/config/iam/format.json/xl.meta
data3/.minio.sys/pool.bin/xl.meta
data3/my-cloud-storage/bigfile.bin/5df985c3-32f2-4544-a171-179952cd9f23/part.1
data3/my-cloud-storage/bigfile.bin/5df985c3-32f2-4544-a171-179952cd9f23/part.2
data3/my-cloud-storage/bigfile.bin/xl.meta
data4/.minio.sys/buckets/my-cloud-storage/.metadata.bin/xl.meta
data4/.minio.sys/config/config.json/xl.meta
data4/.minio.sys/config/iam/format.json/xl.meta
data4/.minio.sys/pool.bin/xl.meta
data4/my-cloud-storage/bigfile.bin/5df985c3-32f2-4544-a171-179952cd9f23/part.1
data4/my-cloud-storage/bigfile.bin/5df985c3-32f2-4544-a171-179952cd9f23/part.2
data4/my-cloud-storage/bigfile.bin/xl.meta
```

Figure 4. Backend metadata and fragment distribution across four disks.

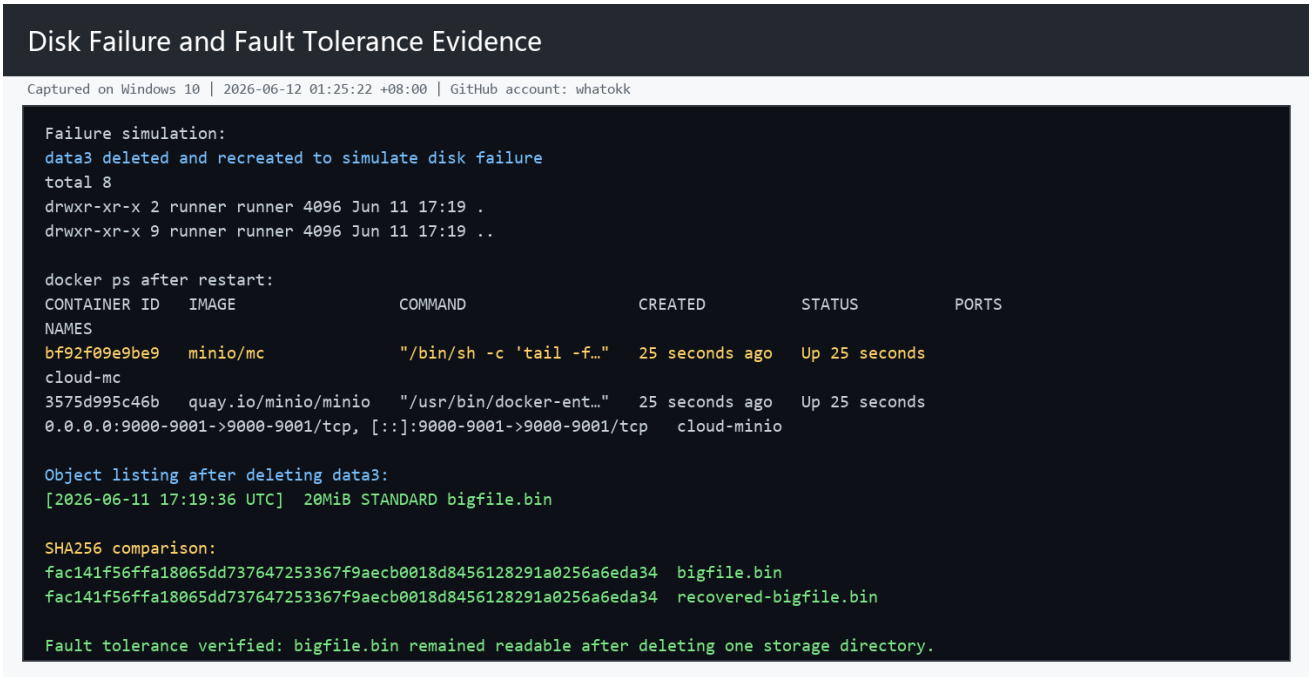

Figure 5. Deleting data3 and verifying fault tolerance.

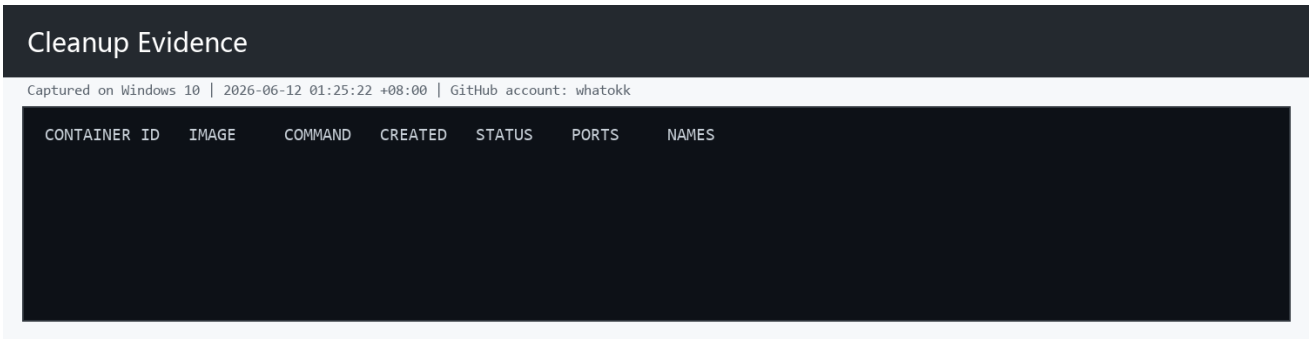

Figure 6. Cleanup verification.

五、Reflection

Problems Encountered and Proposed Solutions: The local Windows Docker daemon was unavailable, so the experiment was executed on a GitHub Actions Ubuntu runner. During the first run, deleting data3 failed because MinIO-created files were owned by container users. The command was corrected to sudo rm -rf data3, matching the lab instruction and allowing the fault simulation to complete successfully.

Why is data still available after deleting one storage directory? MinIO distributed mode uses erasure coding and stores data plus parity information across multiple disks. The object is not stored as one single file on one disk; instead, metadata and fragments are spread across the four storage

directories. When data3 was removed, the remaining disks still contained enough data and parity fragments to reconstruct the object. This is why bigfile.bin was still readable and its recovered SHA256 hash matched the original.

六、Links

Experiment repository: https://github.com/whatokk/lab9-distributed-object-storage

Successful Actions run: https://github.com/whatokk/lab9-distributed-object-storage/actions/runs/27364710248